

RESQNET AI: 14-DAY ENGINEERING SPECS

DETAILED PARALLEL DEVELOPMENT TRACKS & ROLE DISTRIBUTIONS

Target Architecture: Modular 3-Track Python/Edge Engine System
Objective: To distribute the workload efficiently among a 6-person team to ensure completion within a strict 14-day timeline.

To ensure maximum development speed without integration bottlenecks, the project architecture has been split into three completely independent development tracks. Each pair of engineers will work on isolated modules that will be orchestrated together via a master script on Day 12.

TRACK 1: LEAD CORE & AI SYSTEM ENGINEER

HEAVY LOAD TRACK - ASSIGN TO BEST CODER

Primary Assignment: Mathematical Kinematics Filter & Offline Speech Processing

1. Objective & Scope

You are responsible for building the primary computational "brain" of the application. Your work dictates whether an accident is correctly identified and verifies if the driver is conscious. This track carries the highest technical complexity and requires strict processing management.

2. Module A: Vectorized Kinematics & Crash Detection

- **Sensor Interfacing:** Interface with the mobile device's physical tri-axial accelerometer and gyroscope sensors to ingest streaming data at a minimum frequency of 50Hz (50 readings per second).
- **Mathematical Windowing:** Design a system that processes these data packets through a continuous rolling array buffer. Implement the Vector Magnitude Acceleration formula to calculate total G-force impact in real time.
- **Validation Logic:** Establish a multi-stage validation block. An event must only be flagged as a critical collision if the deceleration force exceeds a 4.5G threshold within a tight, continuous time window of less than or equal to 120 milliseconds.
- **False-Positive Rejection:** Develop secondary angular velocity checks from the gyroscope data to verify rapid spatial orientation changes (e.g., vehicular rollover). This distinguishes true impact from a phone simply dropping inside the cabin.

3. Module B: Offline Audio Capture & Speech-to-Text Model

- **Engine Setup:** Configure and embed the open-source Vosk Automatic Speech Recognition (ASR) language model within the local device memory allocation framework. The model footprint must remain strictly restricted under 50MB RAM.

- **Audio Pipeline:** Create an automatic audio recording routine that opens the device microphone channel silently for exactly 15 seconds the moment Module A triggers a crash state.
- **Token Scanning:** Write an asynchronous string-matching scanner that actively looks for predefined cancellation tokens (e.g., "Cancel", "I am safe", "I am okay", "Stop") within the live text conversion stream.
- **Execution State:** If a cancellation keyword is parsed, terminate the emergency sequence instantly. If the 15-second window concludes with zero matching verbal tokens or complete silence, return an absolute "Driver Unresponsive" state code.

TRACK 2: GEOSPATIAL DATA & LOCALIZATION ENGINEER

STANDARD TRACK

Primary Assignment: Embedded Columnar Database Engine & Country-Code Overrides

1. Objective & Scope

Your role is to ensure the application can instantly locate the closest medical facility and adapt to local emergency infrastructure completely offline, anywhere in the world. Your focus is data architecture and ultra-low latency spatial queries.

2. Module A: Pre-Compiled Embedded Spatial Database

- **Schema Architecture:** Design a relational, indexed local database schema using a highly performant in-process database engine (specifically DuckDB).
- **Data Hydration:** Populate the local database storage file with compressed geographic data blocks containing critical care facilities for target test regions (e.g., Egypt and India). Each record must strictly contain: Facility Name, Latitude, Longitude, Cellular/Landline Routing Number, and Trauma Care Readiness Level.
- **Distance Calculations:** Implement a vectorized Haversine mathematical script within the database pipeline. This script must calculate the great-circle distance between the user's current satellite GPS coordinates and every hospital record in the database.
- **Query Optimization:** Optimize database performance using spatial indexing techniques to ensure the "Find Closest Asset" query executes in less than 5 milliseconds natively on the hardware.

3. Module B: Baseband Cellular Country-Code Tracker

- **Hardware Listening:** Write a native telephony interface script that talks directly to the smartphone's baseband radio modem hardware to capture the 3-digit Mobile Country Code (MCC).
- **Dynamic Overrides:** Create a robust, static country mapping engine within the local dataset. For example, if the radio interceptor captures MCC '602', route the application context to Egypt; if it captures '404', route to India.
- **Query Filtering:** Dynamically update the application's target emergency ambulance dialing sequence (e.g., map to 123 for Egypt, 112 for India) and filter all nearest-hospital queries to the detected country partition based on this hardware token.

TRACK 3: FRONTEND UI & SYSTEMS INTEGRATOR

STANDARD TRACK

Primary Assignment: Application Interface, Orchestration Bridge, & Automated SMS

1. Objective & Scope

You are the master builder of the user's physical experience and the final notification pipeline. Your role is to design a high-visibility interface, manage the master coordination loop, and execute the automated emergency communication protocols.

2. Module A: High-Visibility Emergency UI

- **Framework Setup:** Develop the main user interface layout using a modern cross-platform framework (Flutter or React Native).
- **State Management:** Implement a system-wide state visualizer: Normal Monitoring Mode (Green/Passive state) and Active Alert Mode (Flashing Red/High-Priority state).
- **Emergency Overlay:** Design a Countdown screen that forces itself to the front of the device when a crash is detected, displaying a massive, bold 15-second countdown timer.
- **Fallback Overrides:** Create a large, high-contrast, easy-to-tap manual "Cancel False Alarm" screen button that provides a fallback touch override alongside Developer 1's voice verification system.

3. Module B: Master Orchestration & Auto-SMS Alerts

- **The Bridge Script:** Write the master system loop script that acts as the communication pipeline between the frontend UI and the background engines (connecting Dev 1 and Dev 2's modules).
- **Coordinate Pipeline:** Interface with the device's native GPS hardware location chip to pull active latitude and longitude coordinates natively.
- **Execution Flow:** Integrate Developer 1's crash flag. When a true crash occurs, trigger the countdown screen UI and simultaneously signal Developer 1's voice loop. If the verification system returns an unresponsive code, trigger Developer 2's lookup tool to fetch the closest hospital metadata.
- **Transmission Handler:** Build the outbound notification module using the device's native background SMS management API. This module must package the crash metrics, satellite location, and timestamp into a short text message payload (under 160 characters) and automatically dispatch it directly to the hospital's terminal contact number.